

Relatorio Tecnico de Resiliencia

Chaos Engineering com Chaos Mesh

Trabalho Pratico de Sistemas Distribuidos - 2026/1

Universidade Federal do Espirito Santo - UFES / CCENS

Aplicacao: Catalogo de Produtos (API Gateway - Product Service - PostgreSQL)

Data: 01/07/2026

1. Contexto e Arquitetura

Este relatorio documenta os 3 experimentos de caos obrigatorios executados contra a aplicacao de catalogo de produtos: API Gateway (FastAPI, unico ponto exposto) -> Product Service (FastAPI + SQLAlchemy async + asyncpg) -> PostgreSQL, orquestrada em um cluster Kubernetes (Minikube) com 2 replicas por componente.

Mecanismos de tolerancia a falhas testados

- Circuit breaker no gateway: abre apos 5 falhas consecutivas, tenta reconexao apos 15s (half-open).
- Retry com backoff exponencial (tenacity): ate 3 tentativas por chamada ao product-service.
- Timeouts explicitos por fase httpx: connect 2s, read 3s, write 3s, pool 2s.
- Replicas (2 por componente) + HorizontalPodAutoscaler no product-service: min 2, max 5, alvo 50% de CPU.

Ferramentas

- Chaos Mesh (Helm) para os 3 ataques declarativos.
- Prometheus + Grafana (kube-prometheus-stack) para coleta e visualizacao de metricas.
- k6 para geracao de carga continua durante cada ataque (media de 5 usuarios virtuais, 70% GET /products, 20% GET /products/{id}, 10% POST /products).

2. Metodologia

Um unico baseline de 60s (sem nenhum ataque ativo) foi capturado antes dos 3 experimentos, servindo de referencia de 'Estado Estavel' para todos eles. Cada experimento seguiu o ciclo: (1) ~30s de carga sem ataque para contexto local do grafico, (2) aplicacao do manifesto YAML do Chaos Mesh, (3) janela de observacao durante e apos o ataque, (4) remocao do recurso de caos, (5) verificacao de que o sistema voltou ao estado estavel (circuit breaker fechado, replicas prontas) antes de seguir para o proximo experimento. Os 3 experimentos rodaram em sequencia automatizada (scripts em load-test/).

2.1 Estado Estavel (baseline)

Metrica	api-gateway	product-service
Latencia media	9.1 ms	3.9 ms
Taxa de erro (4xx/5xx)	0.0%	0.0%
CPU	33m	34m
Memoria	112 MB	131 MB

Replicas do product-service: 2/2 prontas.

3. Experimento 1 - NetworkChaos (Falha de Rede)

Estado Estavel

Ver secao 2.1 - latencia media de referencia: api-gateway ~9ms, product-service ~4ms, 0% de erros.

Hipotese

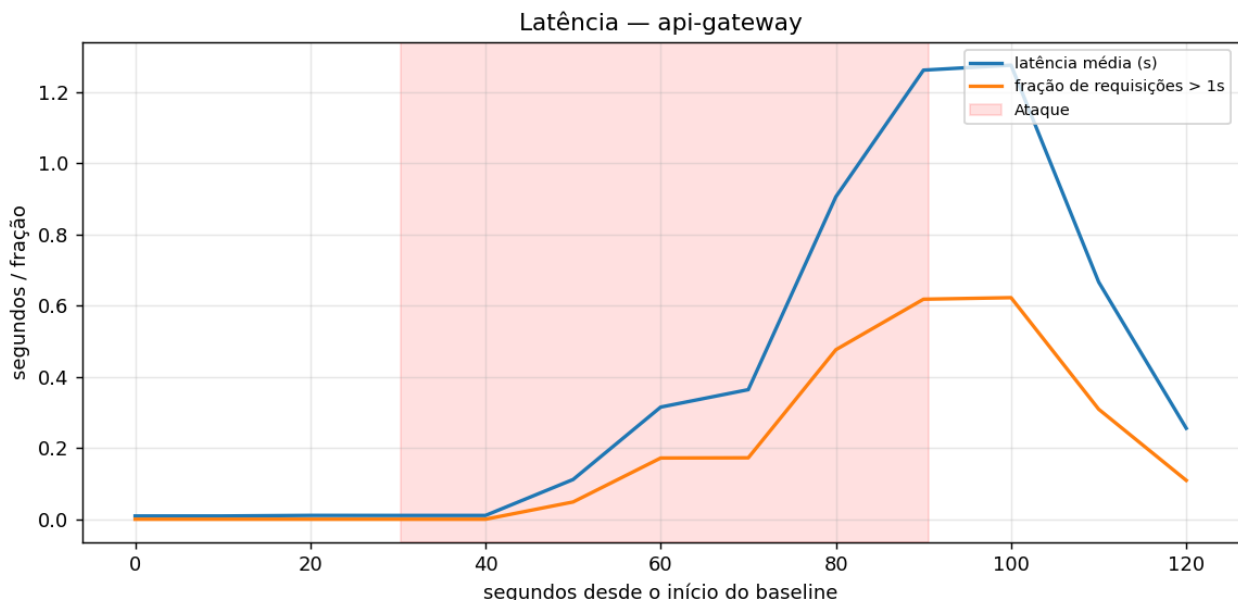
Ao injetar latencia de rede entre product-service e api-gateway, esperamos que a latencia media das requisicoes no gateway aumente proporcionalmente ao delay configurado. Como o delay (2s) fica proximo do timeout de leitura do httpx (3s) mas nao o ultrapassa na maioria dos casos, esperamos que o retry e o circuit breaker absorvam a maior parte do impacto sem expor erros 5xx ao cliente final - ou seja, degradacao graciosa (latencia alta) em vez de indisponibilidade.

Configuracao do Ataque

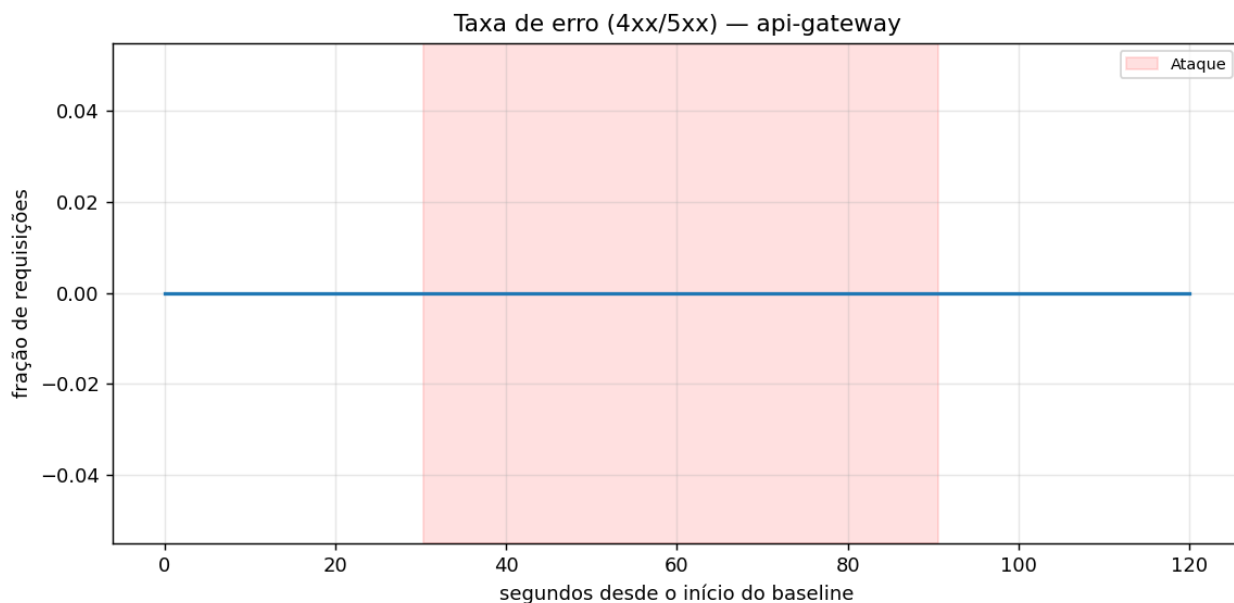
```
NetworkChaos (k8s/chaos/network-chaos.yaml)
  action: delay | selector: app=product-service | target: app=api-gateway, direction: to
  latency: 2s | jitter: 500ms | correlation: 25 | duration: 60s
```

Resultado Observado

Durante os 690 requisicoes geradas pelo k6 ao longo do experimento, a taxa de falha observada pelo cliente foi de 0.0% (latencia media 370ms, p95 2232ms, maximo 3803ms). A latencia media medida no servidor (Prometheus) subiu de ~9ms (baseline) para acima de 1.2s durante e logo apos a janela de ataque, voltando ao normal na sequencia (o pico ocorre um pouco depois do fim do ataque porque a metrica usa uma media movel de 1 minuto).



Latencia media e fracao de requisicoes acima de 1s no api-gateway. A faixa vermelha marca a janela do ataque (60s).



Taxa de erro (4xx/5xx) no api-gateway durante o mesmo periodo - manteve-se proxima de zero.

RESULTADO: latencia impactada significativamente, mas 0% de falhas expostas ao cliente (k6 failed_rate=0.0%) - degradacao

Acoes Corretivas

Os mecanismos ja implementados preventivamente (timeout de leitura de 3s, retry com backoff exponencial e circuit breaker) se mostraram suficientes para este nivel de latencia injetada: nenhuma mudanca de codigo foi necessaria apos o experimento. Como ponto de atencao para trabalhos futuros, um ataque com latencia ainda maior (proxima ou acima do timeout) provavelmente comecaria a acionar o circuit breaker com mais frequencia - esse e o proximo cenario natural a explorar.

4. Experimento 2 - PodChaos (Falha de Instancia)

Estado Estavel

Ver secao 2.1 - 2/2 replicas do product-service prontas, 0% de erros.

Hipotese

Ao matar abruptamente um dos 2 pods do product-service durante carga ativa, esperamos que o Kubernetes detecte a falha rapidamente (poucos segundos) e recrie o pod automaticamente. Como ha 2 replicas e o Service do Kubernetes balanceia entre elas, esperamos que o gateway continue respondendo atraves da replica sobrevivente durante a recriacao, sem necessariamente abrir o circuit breaker.

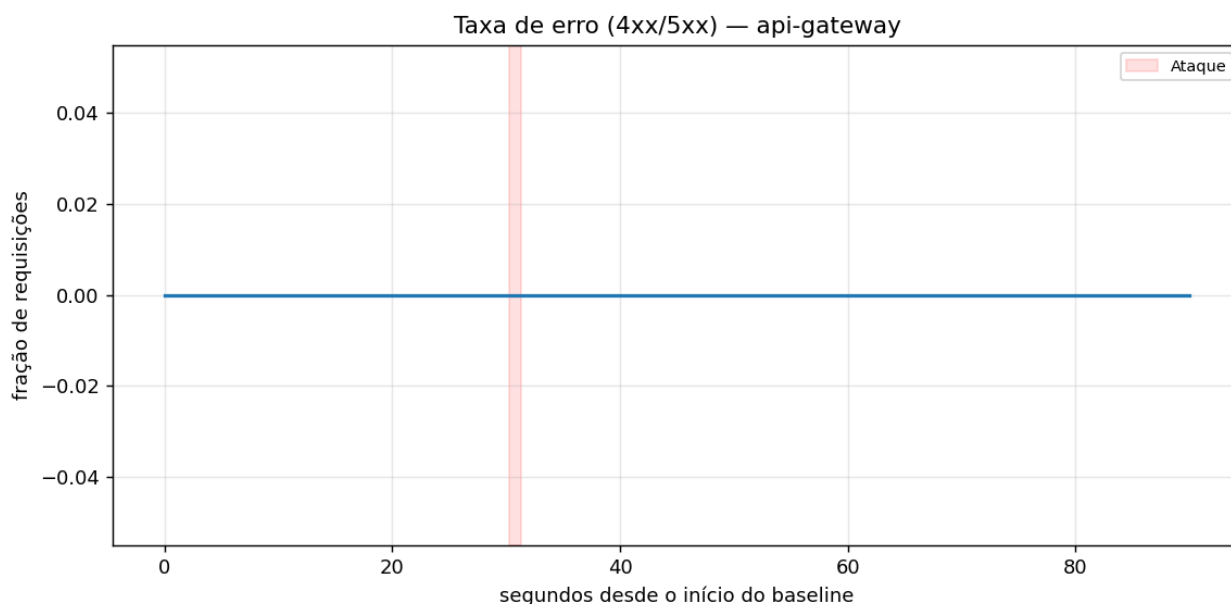
Configuracao do Ataque

```
PodChaos (k8s/chaos/pod-chaos.yaml)
```

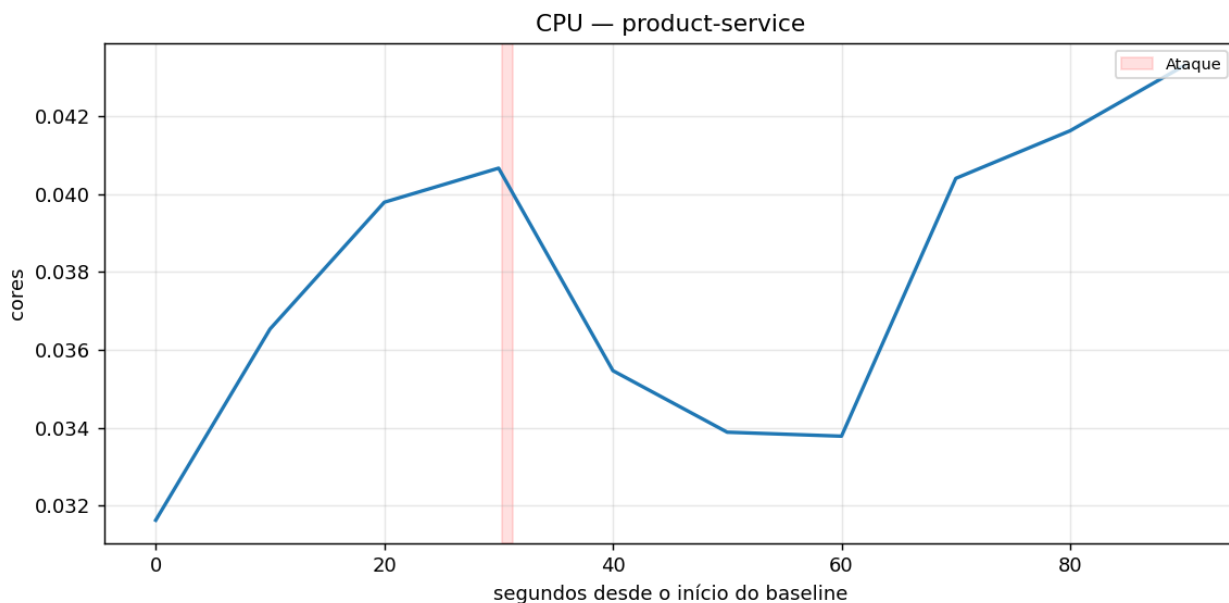
```
action: pod-kill | mode: one | selector: app=product-service | gracePeriod: 0
```

Resultado Observado

Tempo de deteccao pelo Kubernetes: 0.25s. Tempo ate a recuperacao (novo pod Ready): 7.80s. Durante os 870 requisicoes geradas pelo k6, a taxa de falha observada foi de 0.0% (latencia media 18ms, p95 28ms) - a segunda replica absorveu o trafego durante a recriacao do pod, sem degradacao perceptivel no cliente.



Taxa de erro (4xx/5xx) no api-gateway durante o Pod Kill - sem impacto visível.



CPU do product-service durante o experimento (queda visível correspondente a 1 replica menos até a recriação).

RESULTADO: detecção em 0.25s, recuperação total em 7.80s, 0% de falhas expostas ao cliente.

Ações Corretivas

As 2 replicas do product-service e a politica padrao de recriacao de pods do Kubernetes ja eram suficientes para este cenario. Uma acao corretiva relevante ja havia sido aplicada anteriormente, durante os testes de implantacao: o product-service tentava conectar ao PostgreSQL antes do banco estar pronto, causando CrashLoopBackOff no boot. Isso foi corrigido com um retry com backoff exponencial na conexao inicial ao banco (lifespan do FastAPI), o que tambem torna a recriacao do pod apos o Pod Kill mais robusta.

5. Experimento 3 - StressChaos (Falha de Recurso)

Estado Estavel

Ver secao 2.1 - CPU do product-service em repouso: ~34m (34% de um core de 100m requisitado), 2/2 replicas.

Hipotese

Ao injetar sobrecarga de CPU (2 workers a 100% de carga) no product-service, esperamos que a utilizacao de CPU ultrapasse o alvo de 50% configurado no HPA, levando o autoscaler a escalar novas replicas (ate o maximo de 5) para absorver a demanda, mantendo a aplicacao funcional durante o ataque.

Configuracao do Ataque

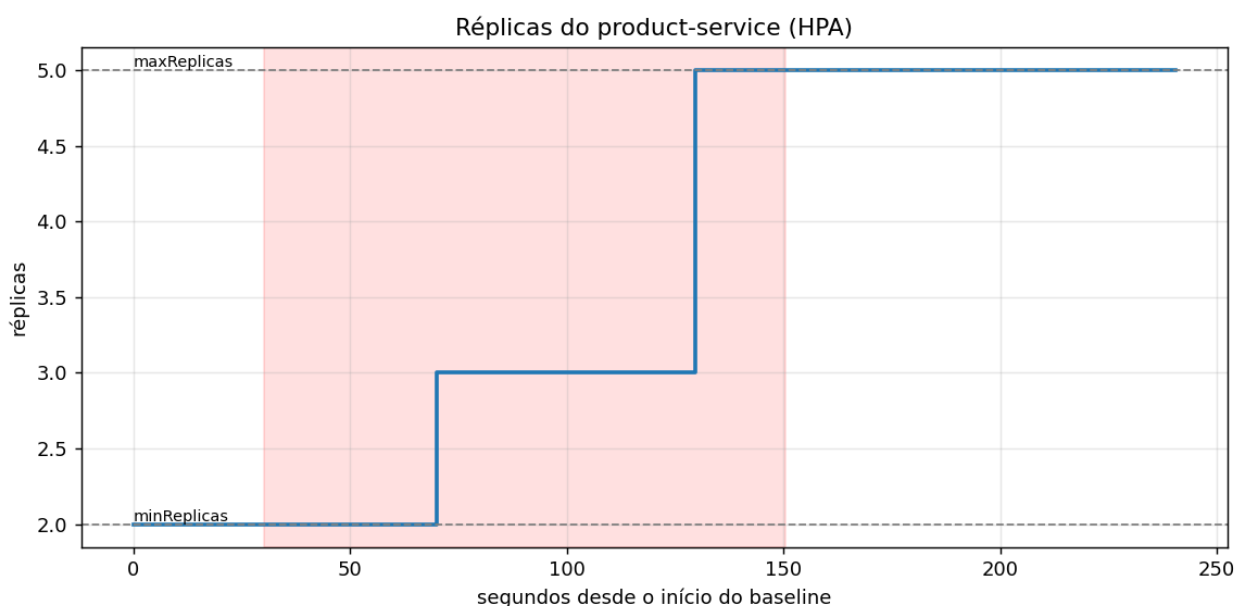
StressChaos (k8s/chaos/stress-chaos.yaml)

```
selector: app=product-service | cpu.workers: 2 | cpu.load: 100 | duration: 120s
```

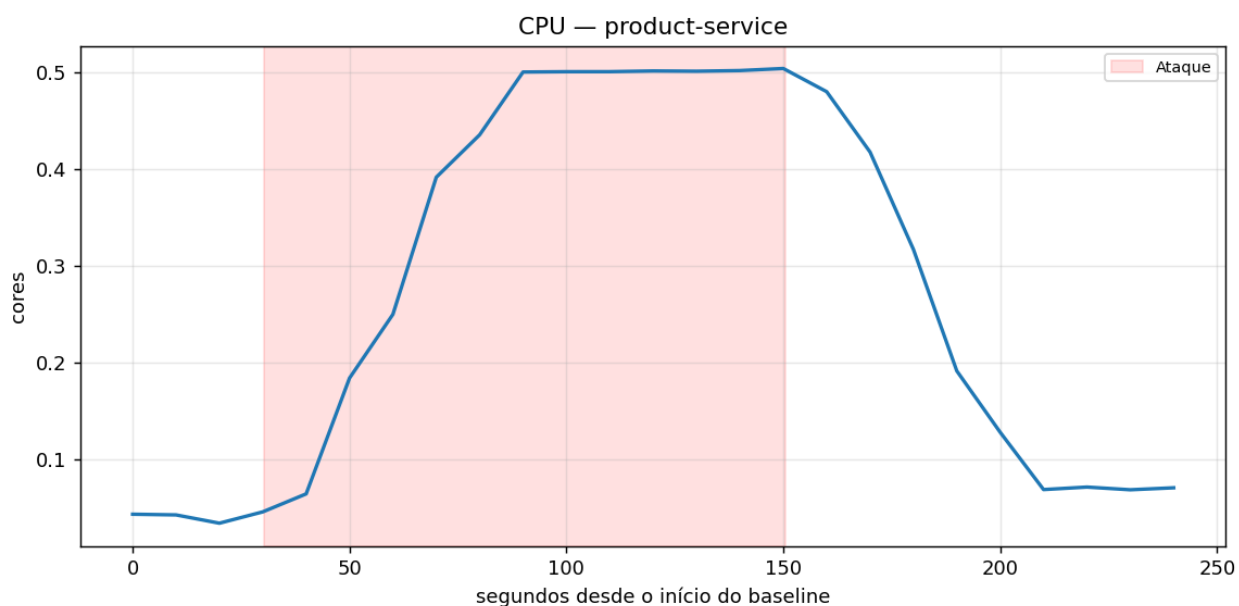
HorizontalPodAutoscaler (product-service-hpa): minReplicas=2, maxReplicas=5, alvo CPU=50%

Resultado Observado

O HPA escalou o product-service de 2 para 5 replicas (o maximo configurado) durante a janela de ataque, permanecendo nesse patamar ate o fim da janela de observacao. Durante os 2100 requisicoes geradas pelo k6, a taxa de falha observada foi de 0.0% (latencia media 71ms, p95 203ms) - a aplicacao se manteve funcional durante toda a sobrecarga.



Réplicas do product-service ao longo do tempo - o HPA escala de 2 para 3 e depois para 5 durante o ataque.



CPU agregada do product-service - ultrapassa o alvo do HPA durante o ataque, disparando o scale-out.

RESULTADO: HPA escalou ate o maximo configurado (5 replicas), 0% de falhas expostas ao cliente.

Acoes Corretivas

O HPA ja configurado (minReplicas=2, maxReplicas=5, alvo 50% CPU) foi suficiente para absorver a sobrecarga sem degradar o servico. Nao foi necessaria nenhuma correcao adicional. Um ponto de atencao para o relatorio: a janela de observacao pos-ataque (90s) nao foi suficiente para capturar o scale-down completo de volta a 2 replicas, dado o stabilizationWindowSeconds de 60s do HPA somado ao tempo para a metrica de CPU cair - o cluster confirmou, em uma verificacao manual posterior, que as replicas voltaram a 2/2 normalmente.

6. Conclusao

Experimento	Metrica-chave	Resultado
NetworkChaos	Latencia media (pico)	9ms -> >1.2s, 0% erros expostos
PodChaos	Deteccao / Recuperacao	0.25s / 7.80s
StressChaos	Replicas (HPA)	2 -> 5 (maximo configurado)

Nos 3 experimentos, a aplicacao nunca ficou indisponivel do ponto de vista do cliente (0% de falhas HTTP observadas pelo k6 em todos os casos) - o sistema degradou de forma graciosa (latencia elevada) ou absorveu a falha via replicas e autoscaling, validando a eficacia dos mecanismos de tolerancia a falhas implementados na camada de software (circuit breaker, timeout, retry) e de infraestrutura (replicas, HPA).